

COM3035 – OPERATING SYSTEMS

FINAL EXAM QUESTIONS – 09.01.2026

1. (20%) The following multithreaded C program uses the POSIX Pthread library. Analyze the code and answer the following questions.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int counter = 0;

void *worker(void *arg) {
    int i = 0;

    for (i = 0; i < 1000; i++) {
        counter++;
    }

    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("%d", counter);

    return 0;
}
```

(a) What will be the final output of this program? Does the output always produce the same result? Explain the underlying reason.

(b) Identify the problem in the code above. Rewrite or modify the code to ensure the program always produces the correct result.

2. (24%) Consider a virtual memory system with a physical memory of 4 frames. The following reference string is given:

7, 0, 1, 2, 0, 3, 4, 0.

Show the state of the frames step by step

- (a) using the FIFO (First-in First-out) algorithm, then find the number of page faults and hits.
- (b) using the LRU (Least Recently Used) algorithm, then find the number of page faults and hits.
- (c) using the Additional Reference Bits algorithm using 8-bit registers, then find the number of page faults and hits.

3. (32%) The following code representation proposes a solution to the *Readers-Writer Problem* using semaphores. Consider the shared variables are defined as:

```
semaphore wrt_mutex = 1;
semaphore mutex = 1;
int read_count = 0;
```

<pre>void writer() { // Code missing... signal(wrt_mutex); }</pre>	<pre>void reader() { wait(mutex); read_count++; if (read_count == 1) wait(wrt_mutex); signal(mutex); // Critical Section (Reading...) wait(mutex); read_count--; if (read_count == 0) signal(wrt_mutex); signal(mutex); }</pre>
---	---

(a) Fill in the missing body of the `writer()` process.

(b) This implementation creates a starvation problem. Explain why this happens based on the synchronization logic and provide an example scenario.

(c) Modify the code to resolve the starvation, ensuring that all three Critical Section Problem solution criterias are met.

4. (24%) Consider the following system resource-process state:

	<u>Allocation</u>				<u>Max</u>				<u>Available</u>			
	A	B	C	D	A	B	C	D	A	B	C	D
P₀	0	0	1	4	0	6	5	6	1	5	2	0
P₁	1	0	0	0	1	7	5	0				
P₂	1	3	5	4	2	3	5	6				
P₃	0	6	3	2	0	6	5	2				

- (a) Explain whether the system is in a safe state or is in a deadlock?
- (b) If P₀ requests for (1, 0, 0, 0), can the request be granted, why?
- (c) If P₂ requests for (1, 0, 0, 0), can the request be granted, why?
- (d) Find and explain the algorithmic time complexity of this deadlock detection system.